VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

AIM:

Implementation of the following Memory Allocation Methods for fixed partition.

a) First Fit

b) Worst Fit

c) Best Fit

ALGORITHM:

Step 1: Start

Step 2: Input

1. Read the number of memory partitions n.
2. Read the size of each partition into the original[] array.
3. Read the number of processes m.
4. Read the size of each process into the processes[] array.

Step 3: First Fit

1. Copy the original partition sizes into the first[] array.
2. Initialize all process allocations to -1.
3. For each process, search the partitions from the beginning.
4. If a partition is free and its size is greater than or equal to the process size:
    - Allocate the process to that partition.
    - Store the partition number.
    - Mark the partition as occupied by setting its size to -1.
    - Stop searching for that process.
5. If no suitable partition is found, mark the process as Not Allocated.
6. Display the First Fit allocation.

PROGRAM:

```
#include <stdio.h>

int main() {

int m, n;

printf("Enter number of memory blocks: ");

scanf("%d", &m);

int blockSize[m];

printf("Enter size of each memory block:\n");

for (int i = 0; i < m; i++) {

printf("Block %d: ", i + 1);

scanf("%d", &blockSize[i]);  }

Printf("Enter number of processes: ");

scanf("%d", &n);

int processSize[n];

int allocation[n];

printf("Enter size of each process:\n");

for (int i = 0; i < n; i++) {

printf("Process %d: ", i + 1);

scanf("%d", &processSize[i]);

allocation[i] = -1;  }

// First Fit

for (int i = 0; i < n; i++) {

for (int j = 0; j < m; j++) {

if (blockSize[j] >= processSize[i]) {

allocation[i] = j;

blockSize[j] -= processSize[i];

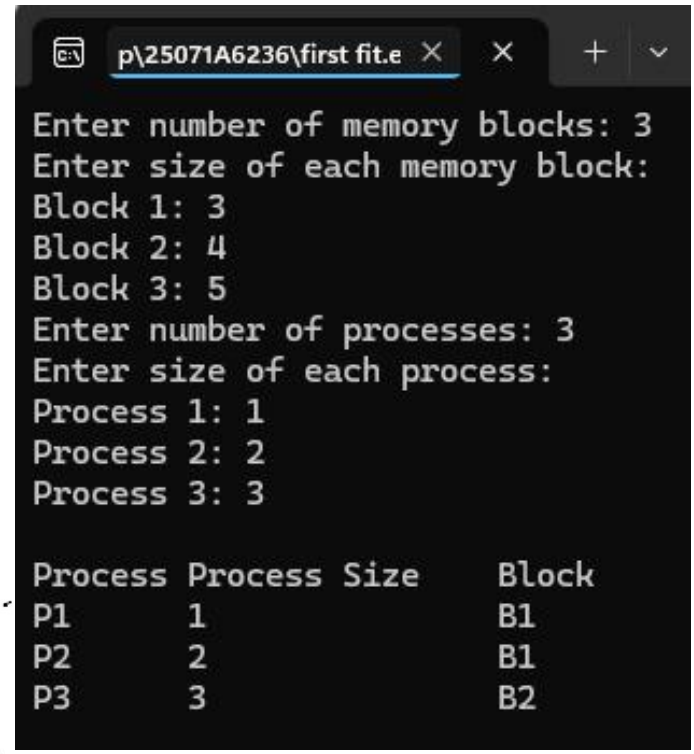
break;

}  } }
```

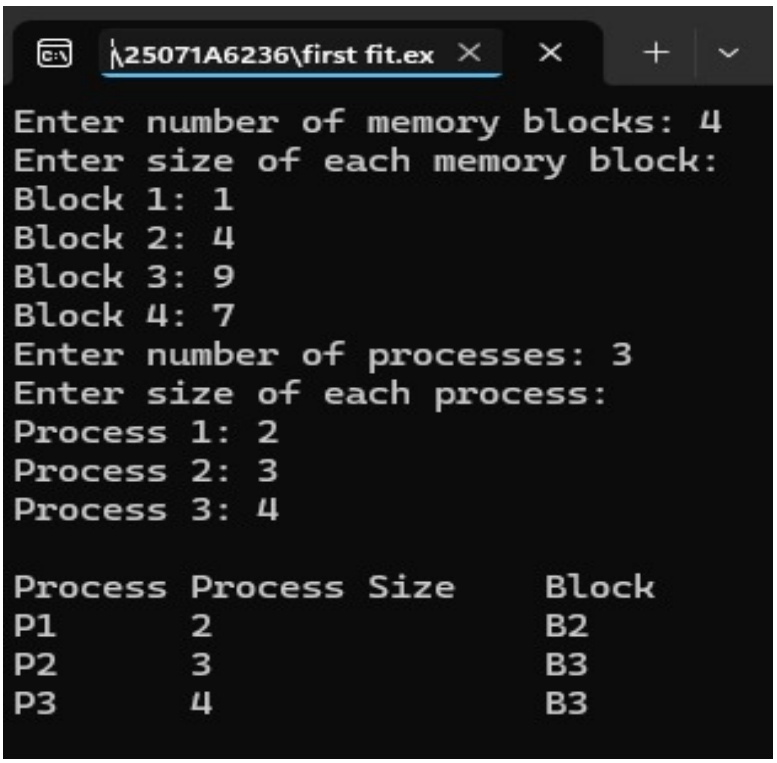
```
// Display result
        printf("\nProcess\tProcess Size\tBlock\n");
        for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t\t", i + 1, processSize[i]);
        if (allocation[i] != -1)
                printf("B%d\n", allocation[i] + 1);
        else
                printf("Not Allocated\n");
        }
        return 0;
        }
```

OUTPUT:

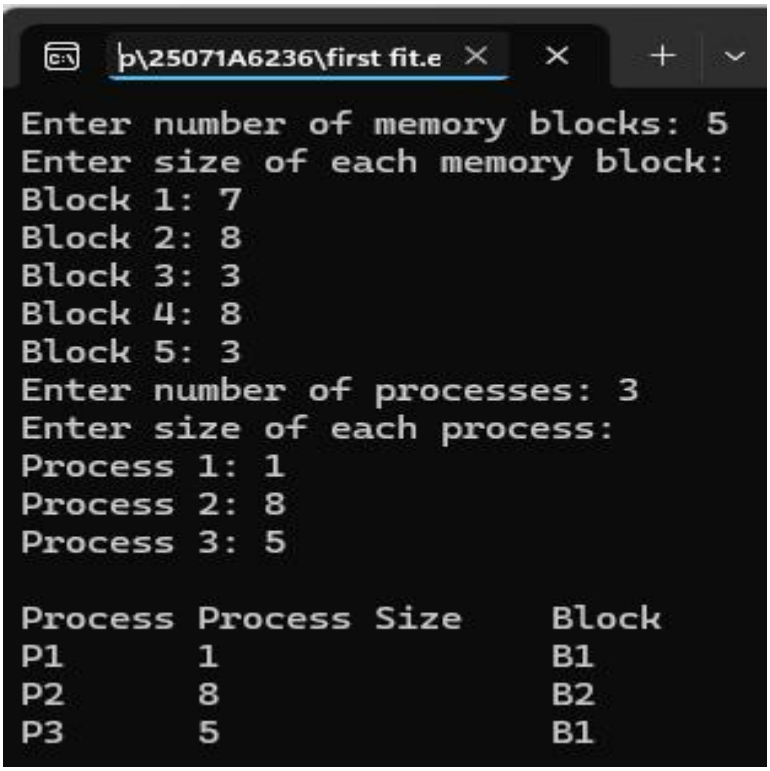
TEST CASE 1:



TEST CASE 2:



TEST CASE 3:



WORST FIT :

Worst Fit allocates a process to the **largest available memory block** that can accommodate it.

Algorithm

1. Start.
2. Read the number of memory blocks m.
3. Read the size of each memory block.
4. Read the number of processes n.
5. Read the memory requirement of each process.
6. For each process:
    - Set worst = -1.
    - Search through all memory blocks.
    - Find the **largest block** whose size is greater than or equal to the process size.
    - Store its index in worst.
7. If worst != -1:
    - Allocate the process to that block.
    - Reduce the block size by the process size.
    - Mark the process as allocated.
8. Otherwise, mark the process as **Not Allocated**.
9. Repeat steps 6–8 for all processes.
10. Display the allocation and remaining memory in each block.
11. Stop.

PROGRAM :

```
#include <stdio.h>

int main() {

    int m, n;

    printf("Enter number of memory blocks: ");

    scanf("%d", &m);

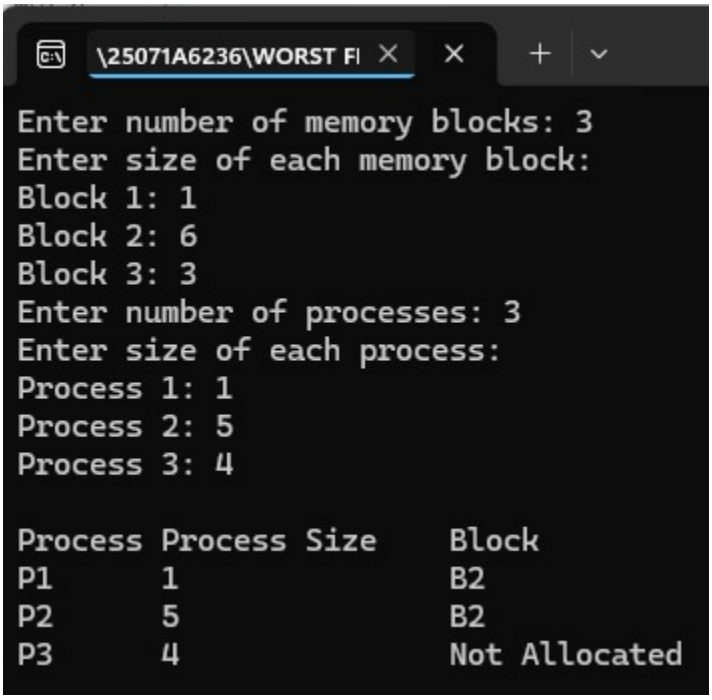
    int blockSize[m];
```

```
printf("Enter size of each memory block:\n");
for (int i = 0; i < m; i++) {
    printf("Block %d: ", i + 1);
    scanf("%d", &blockSize[i]);
}
printf("Enter number of processes: ");
scanf("%d", &n);
int processSize[n];
int allocation[n];
printf("Enter size of each process:\n");
for (int i = 0; i < n; i++) {
    printf("Process %d: ", i + 1);
    scanf("%d", &processSize[i]);
    allocation[i] = -1;
}
// Worst Fit
for (int i = 0; i < n; i++) {
    int worst = -1;
    for (int j = 0; j < m; j++) {
        if (blockSize[j] >= processSize[i]) {
            if (worst == -1 || blockSize[j] > blockSize[worst]) {
                worst = j;
```

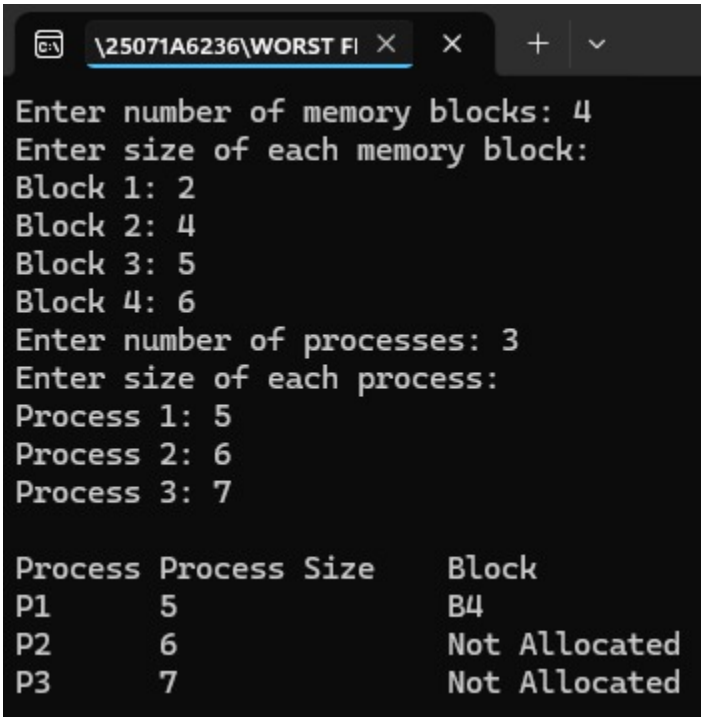
```
            }
        }
    }
    if (worst != -1) {
        allocation[i] = worst;
        blockSize[worst] -= processSize[i];
    }
}
// Display result
printf("\nProcess\tProcess Size\tBlock\n");
for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t\t", i + 1, processSize[i]);
    if (allocation[i] != -1)
        printf("B%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
return 0;
}
```

OUTPUT:

TEST CASE 1 :



TEST CASE 2 :



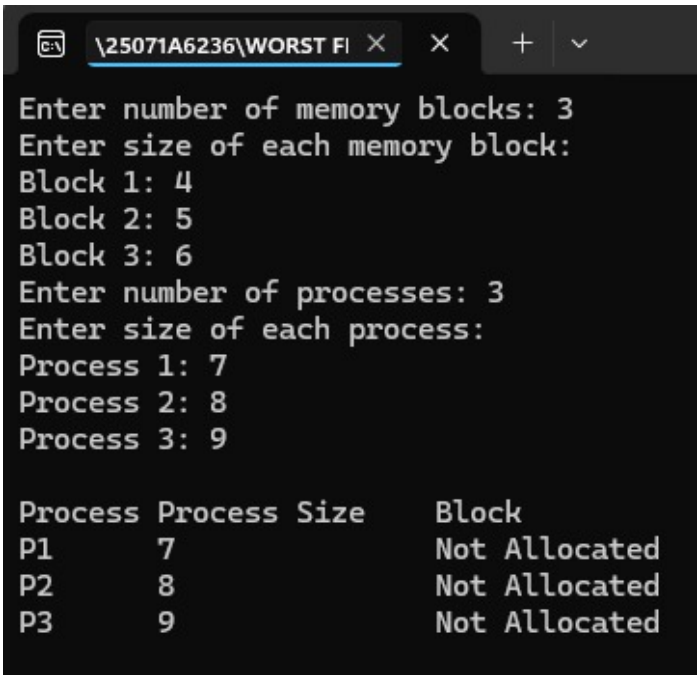
VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

TEST CASE 3 :



BEST FIT :

Best Fit allocates each process to the **smallest available memory block that is large enough**

Algorithm

1. Start.
2. Read the number of memory blocks m.
3. Read the size of each memory block.
4. Read the number of processes n.
5. Read the size of each process.
6. For each process:
    i. Set bestIndex = -1.
    ii. Search through all memory blocks.
    iii. If a block is large enough for the process and is smaller than the current best block, select it.
7. If a suitable block is found:
    i. Allocate the process to that block.
    ii. Reduce the available size of the block by the process size.
    iii. Mark the process as allocated.
8. If no suitable block is found, mark the process as **Not Allocated**.
9. Repeat until all processes are checked.
10. Stop.

VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

PROGRAM :

```
#include <stdio.h>
int main() {
    int m, n;
    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
    int blockSize[m];
    printf("Enter size of each memory block:\n");
    for (int i = 0; i < m; i++) {
        printf("Block %d: ", i + 1);
        scanf("%d", &blockSize[i]);
    }
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int processSize[n];
    int allocation[n];
    printf("Enter size of each process:\n");
    for (int i = 0; i < n; i++) {
        printf("Process %d: ", i + 1);
        scanf("%d", &processSize[i]);
        allocation[i] = -1;
    }
    // Best Fit
    for (int i = 0; i < n; i++) {
        int best = -1;
        for (int j = 0; j < m; j++) {
            if (blockSize[j] >= processSize[i]) {
                if (best == -1 || blockSize[j] < blockSize[best]) {
                    best = j;
                }
            } }
```

```
            if (best != -1) {
                allocation[i] = best;
                blockSize[best] -= processSize[i];
            }
        }

        // Display result
        printf("\nProcess\tProcess Size\tBlock\n");

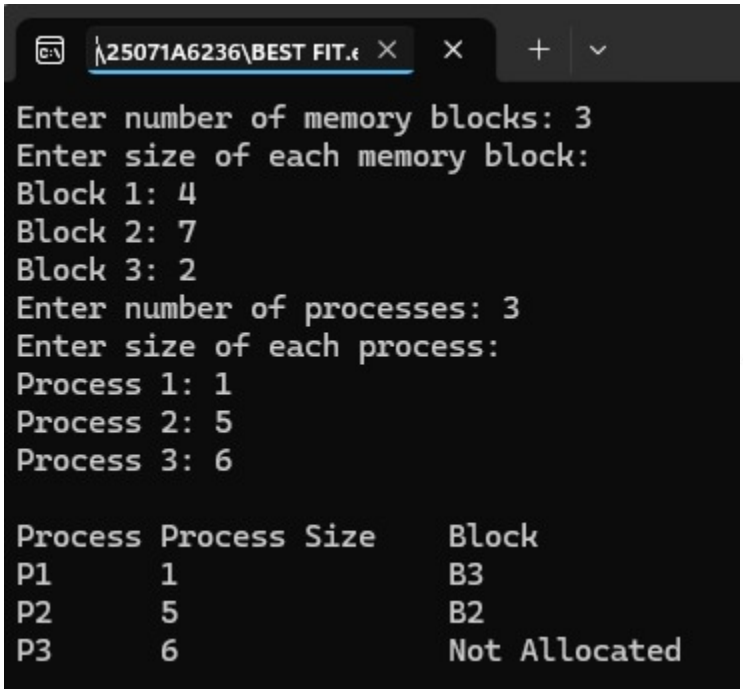
        for (int i = 0; i < n; i++) {
            printf("P%d\t%d\t\t", i + 1, processSize[i]);

            if (allocation[i] != -1)
                printf("B%d\n", allocation[i] + 1);
            else
                printf("Not Allocated\n");
        }

        return 0;
    }
```

OUTPUT :

TSET CASE 1 :



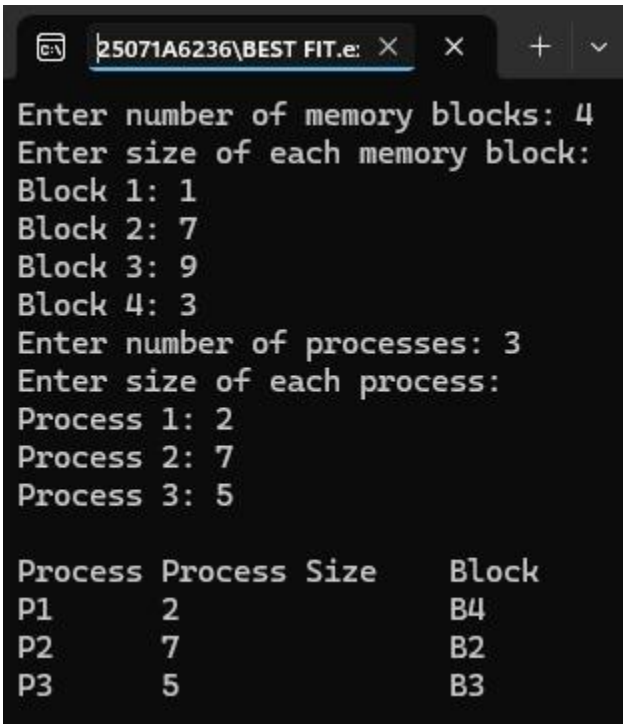
VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

TEST CASE 2 :



TSET CASE 3 :

